Curso Avançado

Capítulo 6 - Controle automático de velocidade por trecho

Selecionar um alvo adequado ao percurso e alterar a solicitação de tração de forma gradual e verificável.

Introdução	O cadastro do capítulo anterior definiu limites. Agora, o programa utiliza o trecho atual e a etapa da operação para escolher um alvo de tração, mantendo a autorização de rota e a ocupação como condições separadas.
Objetivo	Organizar alvos por trecho e uma rampa de solicitação sem confundir comando enviado com velocidade ou parada física.
Meta de aprendizagem	Testar mudanças graduais de solicitação, acompanhar uma sequência de sensores simulados e reconhecer quando a redução precisa começar antes do trecho restrito.

1. Velocidade real e solicitação de tração

Uma solicitação de tração é um comando destinado à interface de controle. A velocidade real é o movimento observado ou medido da composição. A relação entre ambos depende da locomotiva, da carga, da alimentação e do percurso.

Sem medição de movimento, o programa ajusta a solicitação com base nos testes e eventos disponíveis. Ele não confirma que o trem atingiu uma velocidade específica. Um percentual mostrado na tela também não informa quilômetros por hora.

O código deste capítulo é uma simulação no Monitor Serial. Seus percentuais e tempos são fictícios. Não há comando de PWM, central DCC ou motor, e nenhum valor está validado para a EFMN 74.

2. O trecho define uma condição de circulação

Uma região de circulação, uma aproximação e um ponto de parada podem exigir alvos diferentes. Cada perfil precisa indicar o sentido, o local de início da redução e as condições em que foi verificado.

3. Planejar a redução antes da restrição

Se o trem precisa chegar a uma curva ou região restrita com solicitação reduzida, o evento de redução deve ocorrer antes dessa região, com distância suficiente para a resposta da composição nas condições testadas.

Um sensor colocado no início da restrição pode ser tarde demais. O programa não deve considerar que uma mudança de alvo reduz instantaneamente a velocidade. A rampa suaviza o comando, mas a frenagem física depende da locomotiva e do trem.

Perfil didático	Alvo simulado e função
Circulação	50%: acompanhamento do percurso inicial.
Aproximação	25%: alvo menor solicitado antes da região de parada.
Parada	0%: solicitação de tração zero. Ainda exige conferência de parada.

Esses perfis não indicam trechos reais ou posições de sensores da ferrovia. A distância de redução precisa ser determinada por ensaio, considerando a configuração da composição e o espaço disponível.

4. Aplicar os limites do cadastro

O alvo final deve respeitar o menor limite aplicável à locomotiva, à composição e ao trecho. No exemplo, os limites da locomotiva e da composição são 60% e 55%. O perfil do trecho pode impor um valor menor.

Perfil desconhecido, identidade pendente, perda de comunicação ou evento incoerente impede a continuidade normal. Uma solicitação baixa não substitui reserva de rota nem confirmação das agulhas.

5. Rampa de solicitação

Uma rampa altera o comando em pequenos passos ao longo do tempo. Ela permite transições graduais sem prender o programa em uma espera longa. O loop continua disponível para receber eventos e pedidos de emergência.

A rampa deste exercício muda um ponto percentual a cada 100 milissegundos. Esse ajuste serve ao estudo e não define aceleração ou desaceleração física.

6. Etapas do exercício

Etapa	Condição ou ação
BLOCKED	Inicialização ou falha. Solicitação zero e recuperação pendente.
READY	Operador conferiu identidade, composição parada e condições da rota.
CRUISE	Circulação simulada com alvo de 50%.
APPROACH	Evento A aceito. Novo alvo de 25%.
STOPPING	Evento B aceito. Novo alvo zero por rampa.
STOPPED	Comando C aceito após solicitação zero e conferência de parada física.

A e B representam eventos reconhecidos antes dos locais onde seus efeitos são necessários. No código, não há ligação a sensores reais. A sequência deve ser adaptada ao layout e ao sentido da operação.

7. Parada normal e emergência

Na parada normal, o alvo passa a zero e a solicitação diminui gradualmente. A etapa permanece STOPPING até o operador confirmar que a composição realmente parou. Não há liberação automática de trecho.

Na emergência do exercício, a solicitação vai imediatamente a zero e a operação fica bloqueada. Isso é apenas o comportamento da simulação. Na montagem real, o mecanismo de parada depende da interface de tração e da proteção prevista no projeto.

Zero de solicitação não comprova imobilidade e não garante uma distância de parada. As reservas existentes devem permanecer até a situação física ser conferida.

8. Comandos do Monitor Serial

Comando	Função
I / G	Inspecionar condições iniciais / iniciar circulação simulada.
A / B	Solicitar aproximação / solicitar parada normal.
C	Confirmar parada física quando a solicitação já chegou a zero.
X	Emergência: bloquear e zerar a solicitação sem rampa.

9. Código em C++ - estados, limites e decisões

Reúna as duas partes em um único sketch. Use o Monitor Serial em 115200 baud. As mensagens representam solicitações, não medições ou comandos elétricos.

```
enum class State { BLOCKED, READY, CRUISE,
                   APPROACH, STOPPING, STOPPED };
State state = State::BLOCKED;
int requestPercent = 0;
int targetPercent = 0;
const int locomotiveLimit = 60;
const int consistLimit = 55;
const unsigned long rampIntervalMs = 100UL;
const unsigned long stageTimeoutMs = 60000UL;
unsigned long lastRamp = 0;
unsigned long stateStarted = 0;

int minimum(int a, int b) { return a < b ? a : b; }
void enterState(State next) {
  state = next;
  stateStarted = millis();
}
void reportRequest() {
  Serial.print("SIMULATION request: ");
  Serial.print(requestPercent);
  Serial.print("; target: ");
  Serial.println(targetPercent);
}
void setTarget(int trackTarget) {
  targetPercent = minimum(trackTarget,
    minimum(locomotiveLimit, consistLimit));
  lastRamp = millis();
  reportRequest();
}
void blockOperation(const char* reason) {
  requestPercent = targetPercent = 0;
  enterState(State::BLOCKED);
  reportRequest();
  Serial.println(reason);
  // Keep existing route reservations.
}
void handleCommand(char c) {
  if (c == 'X') { blockOperation("Emergency"); return; }
  if (c == 'I' && state == State::BLOCKED) {
    // Manual check: identity, stopped train, profiles,
    // reserved route, turnouts and resources reconciled.
    enterState(State::READY);
    Serial.println("Inspection accepted. Await G.");
  } else if (c == 'G' && state == State::READY) {
    enterState(State::CRUISE);
    setTarget(50);
  } else if (c == 'A' && state == State::CRUISE) {
    enterState(State::APPROACH);
    setTarget(25);
```

10. Código em C++ - rampa e confirmação de parada

```
  } else if (c == 'B' && state == State::APPROACH) {
    enterState(State::STOPPING);
    setTarget(0);
  } else if (c == 'C' && state == State::STOPPING &&
             requestPercent == 0) {
    // Operator confirms physical stop. No resource release.
    enterState(State::STOPPED);
    Serial.println("Stop checked. Reservations retained.");
  } else {
    blockOperation("Unexpected command or event");
  }
}
void updateRamp() {
  if (state == State::BLOCKED || requestPercent == targetPercent)
    return;
  unsigned long now = millis();
  if (now - lastRamp < rampIntervalMs) return;
  lastRamp = now;
  if (requestPercent < targetPercent) ++requestPercent;
  else --requestPercent;
  reportRequest();
}
bool needsTimeout() {
  return state == State::CRUISE || state == State::APPROACH ||
         state == State::STOPPING;
}
void setup() {
  Serial.begin(115200);
  blockOperation("Startup: inspect before I.");
}
void loop() {
  if (needsTimeout() &&
      millis() - stateStarted >= stageTimeoutMs)
    blockOperation("Stage timeout");
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c != '\n' && c != '\r' && c != ' ') handleCommand(c);
  }
  updateRamp();
}
```

Cada atualização altera no máximo um ponto percentual. Se o loop atrasar, a rampa também ficará mais lenta. O código não tenta compensar o atraso enviando vários passos de uma vez.

O prazo de 60 segundos reinicia em cada etapa de movimento e existe somente para o teste. Na operação real, os prazos e distâncias precisam corresponder ao percurso e à composição.

STOPPED encerra o exercício. Para repetir, confira a composição parada, envie X e depois I. A recuperação não retoma a circulação automaticamente e não libera reservas.

11. Roteiro de testes

Teste	Resultado esperado
I G e aguardar	Solicitação sobe gradualmente até 50%.
A após atingir 50%	Alvo muda para 25%. Solicitação diminui por passos.
B após atingir 25%	Alvo zero. Solicitação diminui até zero.
C depois de zero e parada conferida	STOPPED. Reservas mantidas.
B antes de A	Bloqueio por sequência incorreta e solicitação zero.
C antes de solicitação zero	Bloqueio. Confirmação antecipada não é aceita.
X durante a rampa	Solicitação zero imediata, sem aguardar a rampa.
Mais de 60 s sem avançar a etapa	Bloqueio por prazo excedido.

Teste também A enquanto a solicitação ainda sobe. O alvo muda para 25% a partir do valor atual. Se o valor atual estiver abaixo de 25%, o comando ainda poderá subir até esse alvo. A lógica acompanha a solicitação, não uma velocidade medida.

12. Integração com a tração real

A interface de tração deverá receber os comandos permitidos e informar falhas quando aplicável. O coordenador precisa avaliar autorização de rota, identidade, ocupação, confirmação dos desvios e validade dos dados antes de encaminhar o alvo.

Com sensores de passagem, a posição é conhecida em pontos do percurso. Entre esses pontos, a estimativa pode se tornar inadequada. Não use tempo decorrido como confirmação de chegada ou de parada. Uma medição de movimento, quando prevista, exige método e validação próprios.

13. Exercício e registro no Diário de Bordo

Escolha um percurso real e marque circulação, início da redução e local de parada. Registre a composição utilizada, o sentido, a posição dos sensores e o espaço disponível. Deixe os alvos e distâncias como pendentes até os ensaios.

Na simulação, anote alvo, solicitação atual, evento recebido e estado da operação. Nos ensaios futuros, registre também a resposta observada e a distância de parada. A meta é ligar cada mudança de comando a uma condição verificada do percurso.